

Universidad Tecnológica Nacional
Facultad Regional Resistencia
Ingeniería en Sistemas de Información



Elepad

Descripción Técnica

Docentes

- César J. **Acuña**
- Graciela L. **Mendez**

Estudiantes

- Aldo Omar **Andres**
- Sixto Feliciano **Arrejin**
- Agustín Nicolás **Bravo Pérez**
- Tobias Alejandro **Maciel Meister**
- André Leandro **San Lorenzo**

Ciclo Lectivo 2025



Índice

Descripción Técnica.....	3
Base de Datos.....	7
Desarrollo.....	8
Despliegue.....	9
Referencias.....	10



Descripción Técnica

Elepad es una aplicación móvil enfocada en adultos mayores para ayudarlos a aprovechar su día a día y mantenerse conectados con el resto de su familia. El objetivo de cuidar el vínculo con sus familiares también permite a los familiares hacer un seguimiento de sus familiares mayores.

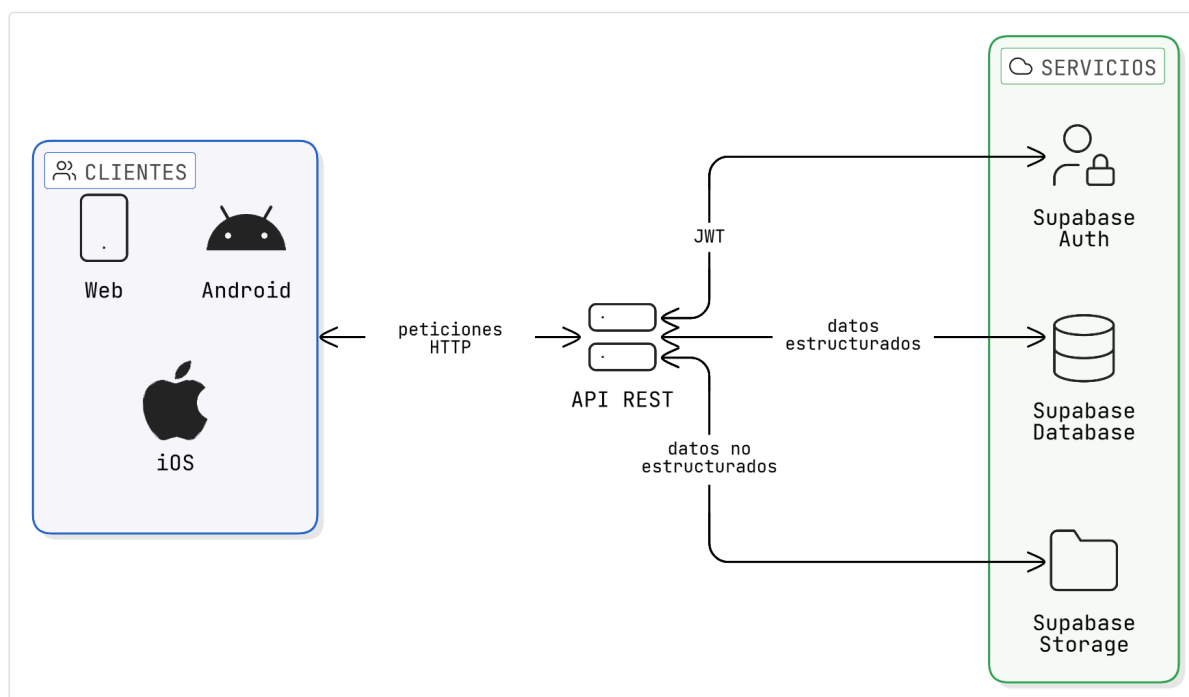
La experiencia de usuario se diseñó para que sea accesible. La aplicación presenta dos roles:

- **Adulto mayor.**
- **Familiar.**

Las secciones importantes de la aplicación son:

- **Inicio:** incluye notificaciones y la configuración del perfil.
- **Calendario:** permite registrar actividades y marcarlas como completadas.
- **Recuerdos:** una galería para compartir fotos, videos y audios con la familia.
- **Juegos:** desafíos lógicos y de memoria para ejercitar la mente. Solo para adultos mayores.
- **Estadísticas:** un historial de los juegos de un adulto mayor, junto con un gráfico que muestra su evolución a lo largo del tiempo. Exclusivo para familiares.

Elepad será una aplicación mobile disponible en iOS y Android, con una versión web menos funcional pero más fácil de acceder. A nivel técnico, Elepad tiene la siguiente arquitectura de software:



Arquitectura de software de Elepad.

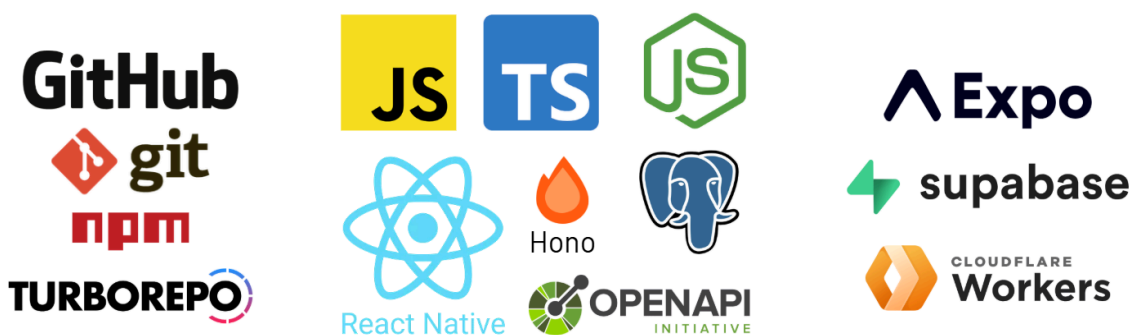
La variedad de clientes justifica una arquitectura cliente/servidor donde el servidor es una API REST stateless. Esta arquitectura se despliega en servicios de la nube, utilizando Expo Application Services para la app web (publicada en <https://ele.expo.app>), Cloudflare Workers para el servidor back end, y Supabase para almacenamiento y autenticación. La autenticación se hace mediante JSON Web Tokens.



El stack de tecnologías incluye:

- **git** como sistema de control de versiones.
- **GitHub** para planificación, pipeline de integración de código y repositorio de código remoto.
- **JavaScript y TypeScript** como lenguaje de programación para el back end y el front end.
- **Hono** como librería para crear servidores y APIs.
- **OpenAPI** como estándar que facilita utilizar herramientas para testear la API.
- **React Native** como framework de UI nativas.
- **React Native Paper** como librería de componentes para la interfaz.
- **Expo v54** como metaframework de React Native que facilita publicar aplicaciones nativas.
- **npm** como gestor de paquetes.
- **nodejs v22** como runtime de JavaScript para el desarrollo.
- **Turborepo** como herramienta para gestionar monorepos de JavaScript.
- **eslint** como linter de JavaScript.
- **Prettier** como formateador automático de código.
- **PostgreSQL** como base de datos relacional.
- **Supabase** como servicio cloud que en su capa gratuita nos ofrece una base de datos PostgreSQL, un servicio de almacenamiento no estructurado (Supabase Storage), y un servicio de autenticación (Supabase Auth).
- **Cloudflare Workers** como nube con una capa gratuita que permite publicar nuestra API REST como un servicio serverless.
- **Google Cloud** para el cliente OAuth que permite a los usuarios iniciar sesión con Google.

En el front end se utiliza Expo como metaframework de React Native. Expo ofrece los siguientes paquetes que resultaron útiles para crear con el mismo código una app que se puede desplegar en web, iOS y Android: expo-notifications, expo-image-picker, expo-media-library, expo-file-system, audio, video, expo-splash-screen, expo-font, expo-keep-awake, expo-web-browser, expo-sharing, expo-clipboard, expo-crypto, expo-screen-orientation, expo-linking, expo-intent-launcher.



Stack de tecnologías de Elepad.

El repositorio de código está organizado como un *monorepo*, es decir que el código de todos los módulos de la aplicación se encuentran dentro del mismo repositorio. Esto facilita ejecutar pipelines de integración y despliegue continuo, y permite a agentes de inteligencia artificial acceder a todo el contexto del proyecto. La estructura del repositorio es la siguiente:



apps/	# Aplicaciones
api/	# Servidor back end con Hono y OpenAPI
mobile/	# App móvil con React Native y Expo
packages/	# Paquetes comunes a las aplicaciones
api-client/	# Hooks de Tanstack Query generados con orval
assets/	# Imágenes de la marca Elepad
supabase/	# Para gestionar Supabase en entorno local

El monorepo cuenta con dos aplicaciones principales:

- **api** es el servidor API REST que funciona como el back end de la aplicación.
- **mobile** es la app nativa que funciona como interfaz de usuario.

Además se tiene un paquete **api-client** que es generado automáticamente a partir del back end, y le permite al front end hacer peticiones HTTP a la API. El paquete **assets** no es muy usado, solamente contiene imágenes de la marca Elepad.

La API REST cuenta con las siguientes rutas principales:

/		
/health		→ Health check
/openapi.json		→ OpenAPI spec (auto-generated)
/users		
GET /users/me		→ Current user profile
PATCH /users/me		→ Update profile
GET /users/:id		→ Get user by ID
/family-groups		
POST /family-groups		→ Create group
GET /family-groups		→ List user's groups
GET /family-groups/:id		→ Get group details
PATCH /family-groups/:id		→ Update group
POST /family-groups/:id/join		→ Join via invite code
DELETE /family-groups/:id/members/:userId		→ Remove member
/activities		
POST /activities		→ Create activity
GET /activities		→ List activities
GET /activities/:id		→ Get activity
PATCH /activities/:id		→ Update activity
DELETE /activities/:id		→ Delete activity
POST /calendar/generate		→ Generate calendar export
/activity-completions		
POST /activity-completions		→ Mark activity complete
GET /activity-completions		→ List completions
/memories		
POST /memories		→ Upload memory
GET /memories		→ List memories



GET	/memories/:id	→ Get memory
PATCH	/memories/:id	→ Update memory
DELETE	/memories/:id	→ Delete memory
/album		
POST	/album	→ Create album
GET	/album	→ List albums
PATCH	/album/:id	→ Update album
DELETE	/album/:id	→ Delete album
/achievements		
GET	/achievements	→ List achievements
POST	/achievements/:id/unlock	→ Unlock achievement
/streaks		
GET	/streaks	→ Get user streak
/shop		
GET	/shop/items	→ List shop items
POST	/shop/items/:id/purchase	→ Purchase item
/notifications		
GET	/notifications	→ List notifications
PATCH	/notifications/:id/read	→ Mark as read
/push-tokens /devices		
POST	/push-tokens	→ Register push token
POST	/devices	→ Register device
/puzzles (public)		
GET	/puzzles	→ List puzzles
GET	/puzzles/:id	→ Get puzzle
/attempts		
POST	/attempts	→ Submit attempt
GET	/attempts	→ List user attempts
/frequencies		
GET	/frequencies	→ List frequencies
POST	/frequencies	→ Create frequency
[sin prefijo] → Público		
/puzzles/* → Público (juegos)		
/attempts/* → Requiere Auth		
Otros → Requieren Auth		

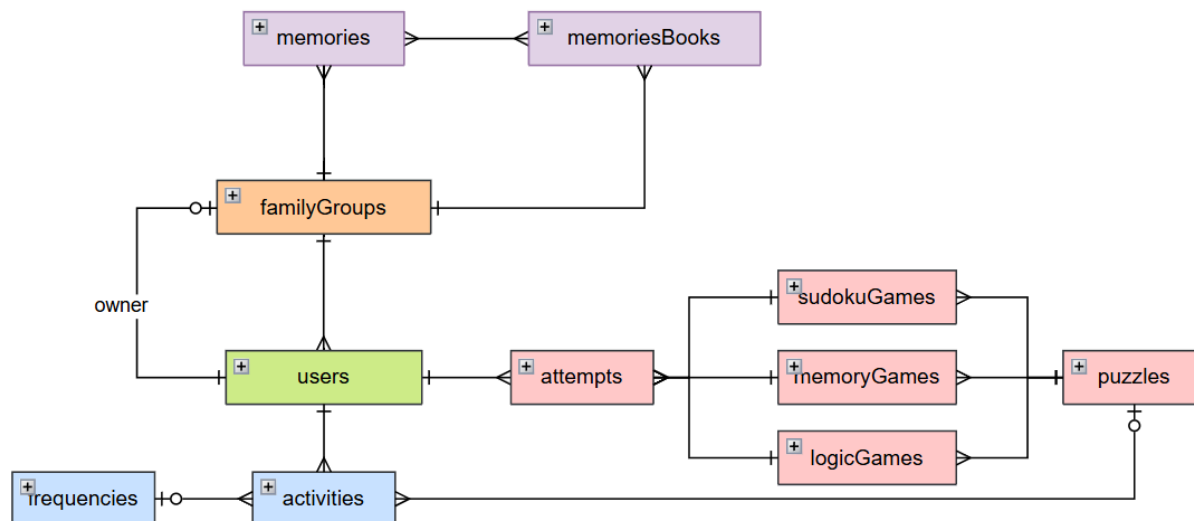
Algunas dependencias importantes del back end:

- pdf-lib: Generación de PDFs.
- ics: Generación de archivos de calendario (.ics).
- @google/genai: Integración con Google Gemini para generar álbumes de recuerdos.



Base de Datos

La base de datos relacional es un PostgreSQL originalmente pensada con la siguiente estructura simple:



DER simplificado de la estructura original de la base de datos.

Conforme se fue desarrollando el proyecto y las historias de usuario fueron cambiando y descubriéndose, este diagrama se expandió para implementar autenticación, notificaciones, más juegos, etc. El listado completo de tablas es el siguiente:

- users - Usuarios de la aplicación.
- familyGroups - Grupos familiares.
- memoriesBooks - Libros de recuerdos.
- memories - Recuerdos individuales.
- frequencies - Frecuencias para actividades.
- activities - Actividades.
- puzzles - Puzzles/juegos.
- sudokuGames - Juegos de sudoku.
- memoryGames - Juegos de memoria.
- logicGames - Juegos de lógica (Lights Out).
- attempts - Intentos de juegos.
- push_tokens - Tokens para notificaciones push.
- device_tokens - Tokens de dispositivo.

Esta base de datos se despliega en la capa gratuita de Supabase, un servicio cloud que también ofrece un servicio de almacenamiento de archivos no estructurados (Supabase Storage) donde guardamos las imágenes y videos de los recuerdos, y un servicio de autenticación (Supabase Auth) que utilizamos para el inicio de sesión con Google. Supabase ofrece un dashboard web muy conveniente desde el cual gestionar la base de datos, aunque cualquier cliente PostgreSQL funciona correctamente.



Desarrollo

Se aplicaron técnicas ágiles y prácticas DevOps para trabajar de manera productiva y asíncrona. Los cinco miembros del equipo se mantuvieron en comunicación constante durante el desarrollo de Elepad, con reuniones periódicas para integrar código y organizar el trabajo pendiente.

Todas las historias de usuario se registraron como un Issue en GitHub Projects, lo cual es fácil de referenciar desde una Pull Request. Esto facilita el flujo de trabajo de los desarrolladores ya que no tienen que irse a otra app como Jira, y facilita el seguimiento de la planificación ya que la información registrada es más parecida a la realidad.

Los cinco miembros del equipo somos desarrolladores de software con experiencia en React web, por lo que desarrollar una aplicación nativa con React Native fue la manera más sencilla de ejecutar el proyecto. Dado que todos los integrantes son desarrolladores full stack, las tareas se dividieron por historias de usuario y no por capas de la aplicación, es decir que no existió un "responsable del back end" o "programador front end", sino que todo el código era responsabilidad de todo el equipo.

Se utilizó una pipeline de integración continua para mantener la calidad del código. Esta pipeline se dispara en cada commit a la rama *main* y contiene los siguientes pasos:

1. Instalar dependencias.
2. Ejecutar el linter (testing estático de código).
3. Ejecutar el build.

El siguiente gráfico muestra el aumento de la dedicación de esfuerzo al proyecto una vez terminado el cursado de las asignaturas de 2025, dado que la mayoría de integrantes del equipo estaban listos para recibirse una vez terminado el proyecto final.



Commits a lo largo del año en el repositorio de Elepad.



Despliegue

El despliegue de Elepad se hace de manera automática mediante una GitHub Action que despliega el back end a Cloudflare Workers y el front end a Expo Application Services. Esta pipeline se puede disparar en cualquier momento, y normalmente se hacía al final de cada sprint o cada vez que se corregía un error serio.

Elepad se ha desplegado en capas gratuitas de diversos servicios cloud para evitar gastos durante el desarrollo del proyecto. También se ha publicado la app en la Google Play Store para que cualquier usuario pueda descargarla. Despliegues:

- **Android:** <https://play.google.com/store/apps/details?id=com.elepadorg.elepad>. Publicado en la Google Play Store.
- **Web:** <https://ele.expo.app/>. Publicada en Expo Application Services.
- **API:** <https://api.elepad.workers.dev/>. Publicada en Cloudflare Workers. Nota: no se puede ver la documentación de OpenAPI debido a que el despliegue es serverless y entonces no está disponible el archivo JSON a partir del cual se genera esa página.
- **Landing page:** <https://elepad.vercel.app/>. Publicada en Vercel.
- **Base de datos:** <https://supabase.com/dashboard/project/sdnmoweppzszpxyggdyg> (privado). Desplegado en Supabase.

Estos despliegues no tienen costos recurrentes debido al mínimo tráfico de la aplicación, pero eventualmente será necesario cancelar la suscripción a los servicios si Elepad no se utiliza, o comenzar a pagar mejores servicios para soportar un tráfico mayor.



Referencias

Especificación de Requisitos de Software (ERS).

https://docs.google.com/document/d/1R3vB02NTxqxi9H_KYEbNzvEl6xEbmV-Q1nAyWVGubfI

Tablas de la Planificación.

<https://docs.google.com/spreadsheets/d/1glGEhmliuJBsaTKYBOxELOV4UhCdjYIOA-YGr7n7PVI>

Repositorio en GitHub.

<https://github.com/elepad-org/Elepad>

Backlog del Proyecto en GitHub.

<https://github.com/orgs/elepad-org/projects/2/views/3>

Roadmap del Proyecto en GitHub.

<https://github.com/orgs/elepad-org/projects/2/views/2>

Despliegue Web de Elepad.

<https://ele.expo.app>

Página de Elepad en la Google Play Store.

<https://play.google.com/store/apps/details?id=com.elepadorg.elepad>